



ALGORITHMS

RSA-DH-MAC



R S A P U B L I C - K E Y E N C R Y P T I O N A L G O R I T H M

- One of the first, and probably best-known public-key scheme
- It was developed in 1977 by R.Rivest, A.Shamir and L. Adleman
- RSA is a block cipher in which the plaintext and ciphertext are **integers** between **0** and **$n-1$**
- Every integer can be represented, of course, as a sequence of bits

Basic Math Concepts I

- An integer $n \geq 2$ is said to be **prime** if the only divisors of n are 1 and n .
- Let a and b be integers (not both zero). The **greatest common divisor** of a and b , denoted $\gcd(a, b)$, is the *largest integer* that divides both a and b . For example, $\gcd(4, 6) = 2$, $\gcd(4, 9) = 1$, $\gcd(15, 25) = 5$
- If $\gcd(a, b) = 1$, then we say that a and b are **relatively prime**.
- Euclid's algorithm is a method to find the gcd of two integers a and b .
- **Theorem:** If $c = \gcd(a, b)$, there exist integers x and y such that $c = x \cdot a + y \cdot b$. For example, $(-1, 1)$, $(-2, 1)$, and $(2, -1)$ in the examples above.

Basic Math Concepts II

- One can modify Euclid's algorithm to find these numbers x and y while computing $\gcd(a, b)$. This is the so-called "Extended Euclidean algorithm".
- For $b > 0$, the modulo operator, denoted by $a \bmod b$, defines the remainder r of a when divided by b . In other words, r is always an integer in the set $\{0, 1, 2, \dots, b - 1\}$.
- Sometimes it is convenient to talk about congruence modulo n .
If $a \bmod n = b \bmod n$, we say that a is congruent to b modulo n and write $a \equiv b \pmod{n}$.

For example, $27 \bmod 5 = 2$, $12 \bmod 3 = 0$, $91 \bmod 10 = 1$

ENCRYPTION AND DECRYPTION IN RSA

- **Encryption:** A message block $M < n$ (represented as a number) is encrypted as

$$C \equiv M^e \bmod n$$

- **Decryption:** To retrieve M , compute

$$M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$$

Here M is a block of plaintext, C is a block of a ciphertext, and e and d are some numbers. Sender and receiver know n and e . Only the receiver knows the value of d .

KEY GENERATION

- Select two prime numbers p and q
- Calculate $n = p * q$
- Calculate $\phi(n) = (p - 1) * (q - 1)$
- Select integer e less than $\phi(n)$ and relatively prime with $\phi(n)$
i.e., $\gcd(e, \phi(n)) = 1$
- Calculate d such that: $de \equiv 1 \text{ mod } \phi(n)$
(using the Extended Euclidean Algorithm: find integers x, y such that $ex + \phi(n)y = 1$,
and d is the modular inverse of e modulo $\phi(n)$)
- Public key $KU = \{e, n\}$ and Private key $KR = \{d, n\}$

FERMAT – EULER THEOREM

Fermat's Little Theorem:

$$x^{p-1} \equiv 1 \pmod{p}$$

where p is a prime number *and* $x \not\equiv 0 \pmod{p}$

Correctness of RSA can be proved
using Fermat-Euler theorem:

$$x^{\phi(n)} \equiv 1 \pmod{n}$$

Where x is relatively prime to $n \geq 2$.

Why Does Decryption Work?

Using Key Relationship:

Since $de \equiv 1 \pmod{\phi(n)}$, we can express it as:

$$de = k\phi(n) + 1 \text{ for some integer } k$$

Applying Fermat-Euler's Theorem:

$M^{\phi(n)} \equiv 1 \pmod{n}$, so we simplify:

$$M^{de} \equiv M^{k\phi(n)+1} \equiv M^{k\phi(n)} * M \equiv (M^{\phi(n)})^k * M \equiv 1^k * M \equiv M \pmod{n}$$

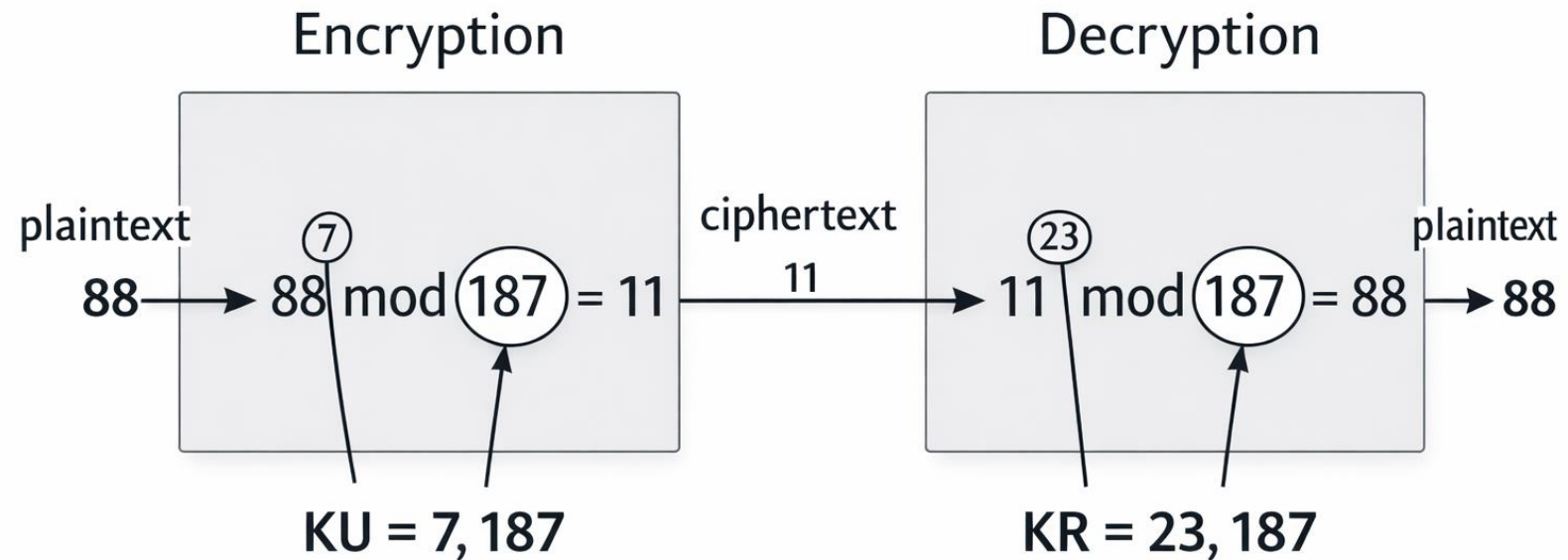
Hence, decryption correctly retrieves the original message block M .

EXAMPLE

- Select two prime numbers, $p = 17, q = 11$
- Calculate $n = pq = 187$
- Calculate $\phi(n) = 16 * 10 = 160$
- Select e less than 160 and relatively prime with 160, for example 7
- Determine d such that $de \bmod 160 = 1$ and $d < 160$
- The correct value is $d = 23$, indeed $23 * 7 = 161 = 1 \bmod 160$
- Thus $KU = \{7, 187\}$ and $KR = \{23, 187\}$ in that case

ENCRYPTION AND DECRYPTION

Let a plaintext be $M = 88$; then encryption with a key $\{7, 187\}$ and decryption with a key $\{23, 187\}$ go as follows



HOW TO BREAK RSA



Brute-force approach: try all possible private keys of the size n . Too many of them even for moderate size of n .



More specific approach: given a number n , try to find its two prime factors p and q ; knowing these would allow us to find a private key easily.

SECURITY OF RSA

Relies upon complexity of factoring problem

Nobody knows how to factor the big numbers in the reasonable time (say, in the time polynomial in the size of (binary representation of) the number

On the other-hand nobody has shown that the fast factoring is impossible

R S A C H A L L E N G E

- RSA Laboratories to promote investigations in security of RSA put a challenge to factor big numbers (starting 1991)
- Least number, not yet factored in that challenge is
- RSA-260 =
 - 22112825529529666435281085255026230927612089502470015394413
74831912882294140200198651272972656974659908590033003140005
11707422045608592763579537571859542988389587092292384910067
03034124620545784566413664540684214361293017694020846391065
875914794251435144458199
- 862 bits, or 260 decimal digits

RSA CHALLENGE, RECENT NEWS (~2020)

RSA-250 =

2140324650240744961264423072839333563008614715144755017797754920881
4180234471401366433455190958046796109928518724709145876873962619215
5736304745477052080511905649310668769159001975940569345745223058932
5976697471681738069364894699871578494975937497937

=

641352894770715802787901901705773890848250147429434472081168596
32024532344630238623598752668347708737661925585694639798853367

X

3337202759497815655622601060535511422794076034476
755466678452098702384172921003708025744867329688187756571898625
8036932062711

HOW TO BREAK RSA (CONT.)

Common factors attack (2012):

- Due to insufficiently good random number generators used in key generation, some number of keys used in the wild have common divisors – you can then factorize them using Euclid's algorithm very fast. But it is not targeted.
- Shor's factorization algorithm for quantum computers (near future?)
 - requires a lot of “high-quality” universal qubits
 - era of post quantum cryptography

POPULAR



Most Public Key Infrastructure (PKI) use RSA



SSL/TLS: Certificates and Key-exchange



Secure e-mail



DIFFIE-HELLMAN KEY EXCHANGE

D I F F I E - H E L L M A N K E Y E X C H A N G E



Most popular algorithm for key exchange is Diffie-Hellman algorithm (1976)



The purpose of the algorithm is exchange of a secret key (not encryption)



DH algorithm is considered as a public-key algorithm because:



Users to generate the same secret key rely on publicly known information + some private information



In principle, it is possible to generate a secret key knowing only public information, but it is computationally expensive

DISCRETE LOGARITHMS

- Security of DH algorithm relies upon difficulty of computing discrete logarithms.
- It is relatively easy calculate exponentials modulo a prime,
that is given a, i, p : Calculate $a^i \bmod p$
- It is very difficult, and for large primes *infeasible*, to calculate discrete logarithms,
that is given b, a, p : Find i such that $b \equiv a^i \bmod p$

DISCRETE LOGARITHMS

A number a is called a **primitive root** if its powers generates all non-zero $b \bmod p$, i.e., for every $0 < b < p$ we have $a^i \equiv b \bmod p$ for some integer i

For example, 3 is a primitive root mod 7 but not 2

Check $a = 2$

$$2^1 = 2 \equiv 2 \bmod 7$$

$$2^2 = 4 \equiv 4 \bmod 7$$

$$2^3 = 8 \equiv 1 \bmod 7$$

Now it repeats:

$$2^4 \equiv 2,; 2^5 \equiv 4,; 2^6 \equiv 1 \bmod 7$$

Check $a = 3$

$$3^1 = 3 \equiv 3 \bmod 7$$

$$3^2 = 9 \equiv 2 \bmod 7$$

$$3^3 = 27 \equiv 6 \bmod 7$$

$$3^4 = 81 \equiv 4 \bmod 7$$

$$3^5 = 243 \equiv 5 \bmod 7$$

$$3^6 = 729 \equiv 1 \bmod 7$$

The set of residues produced is: $\{1,2,3,4,5,6\}$

D I F F I E - H E L L M A N K E Y E X C H A N G E

- Two publicly known numbers:
prime number q
primitive root α of q
- Let A and B wish to exchange a key, then they do the following:

A selects a random integer $X_A < q$ and keeps it in secret
B selects a random integer $X_B < q$ and keeps it in secret

THE SECRET KEY

- Exchange:

A computes $Y_A = \alpha^{X_A} \bmod q$ and sends it to B

B computes $Y_B = \alpha^{X_B} \bmod q$ and sends it to A

- Both A and B is now able to calculate common secret key:

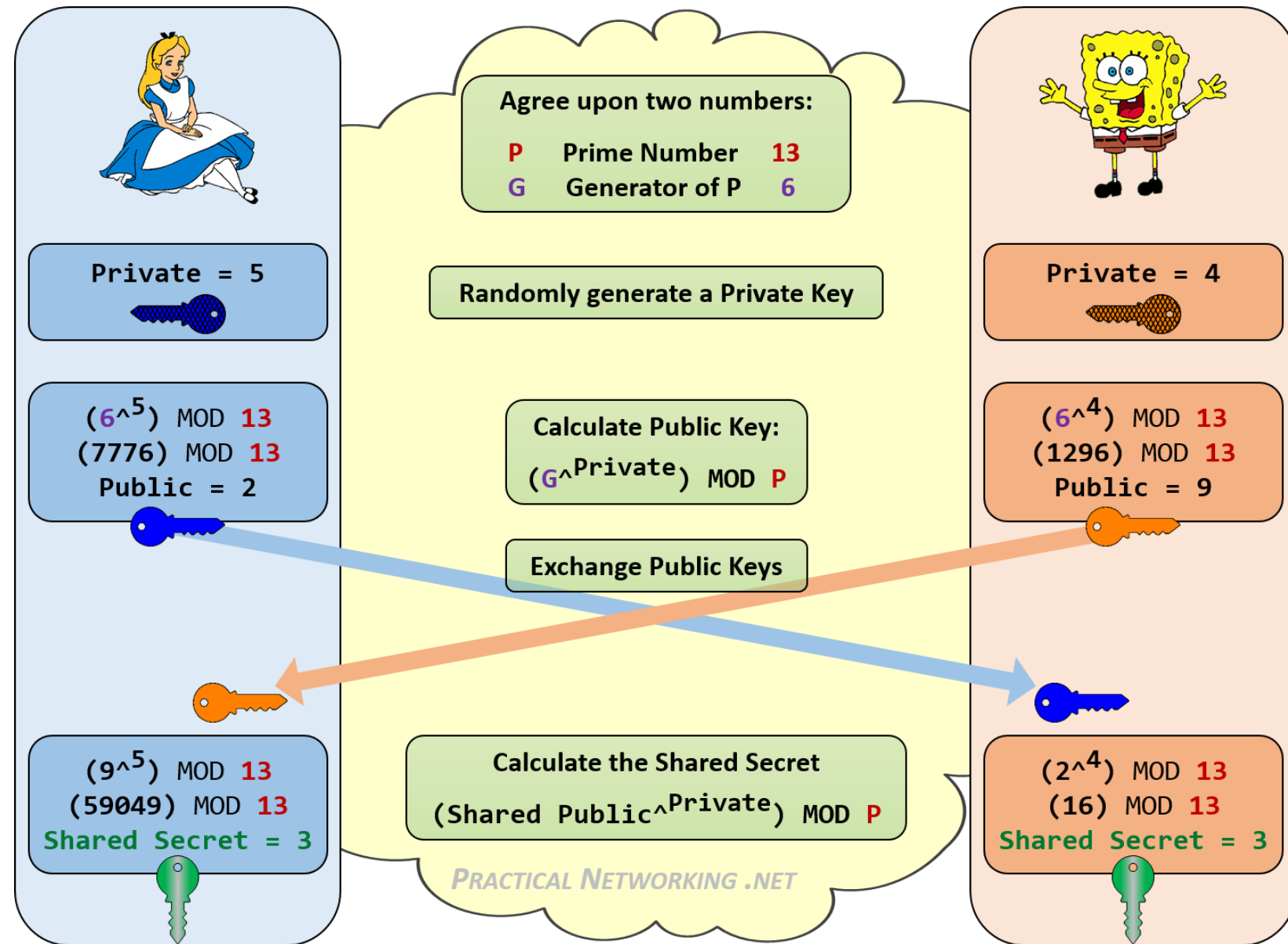
A calculates $K = (Y_B)^{X_A} \bmod q$

B calculates $K = (Y_A)^{X_B} \bmod q$

- These calculations give identical results, and $K = (\alpha)^{X_A X_B} \bmod q$ is the common secret key.

DEFFIE- HELLMAN

KEY EXCHANGE



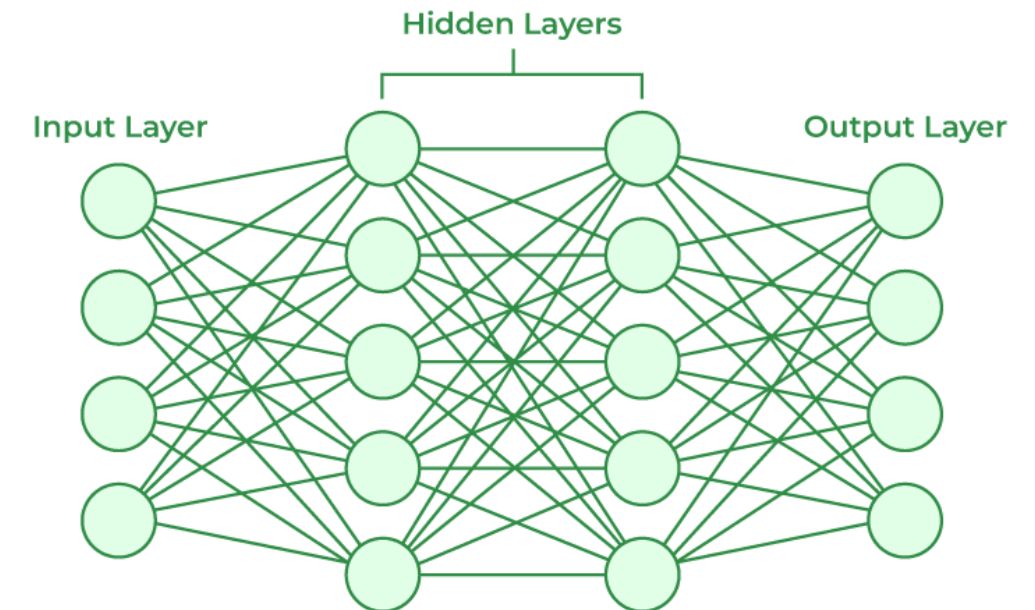
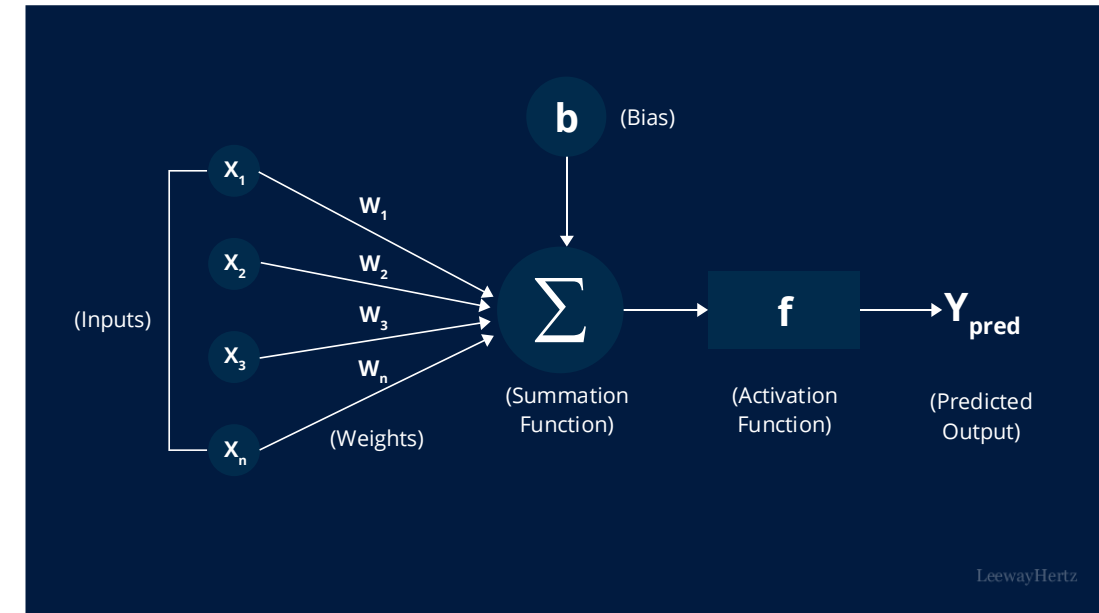
Neural Key Exchange Protocol (NKEP)

What is NKEP?

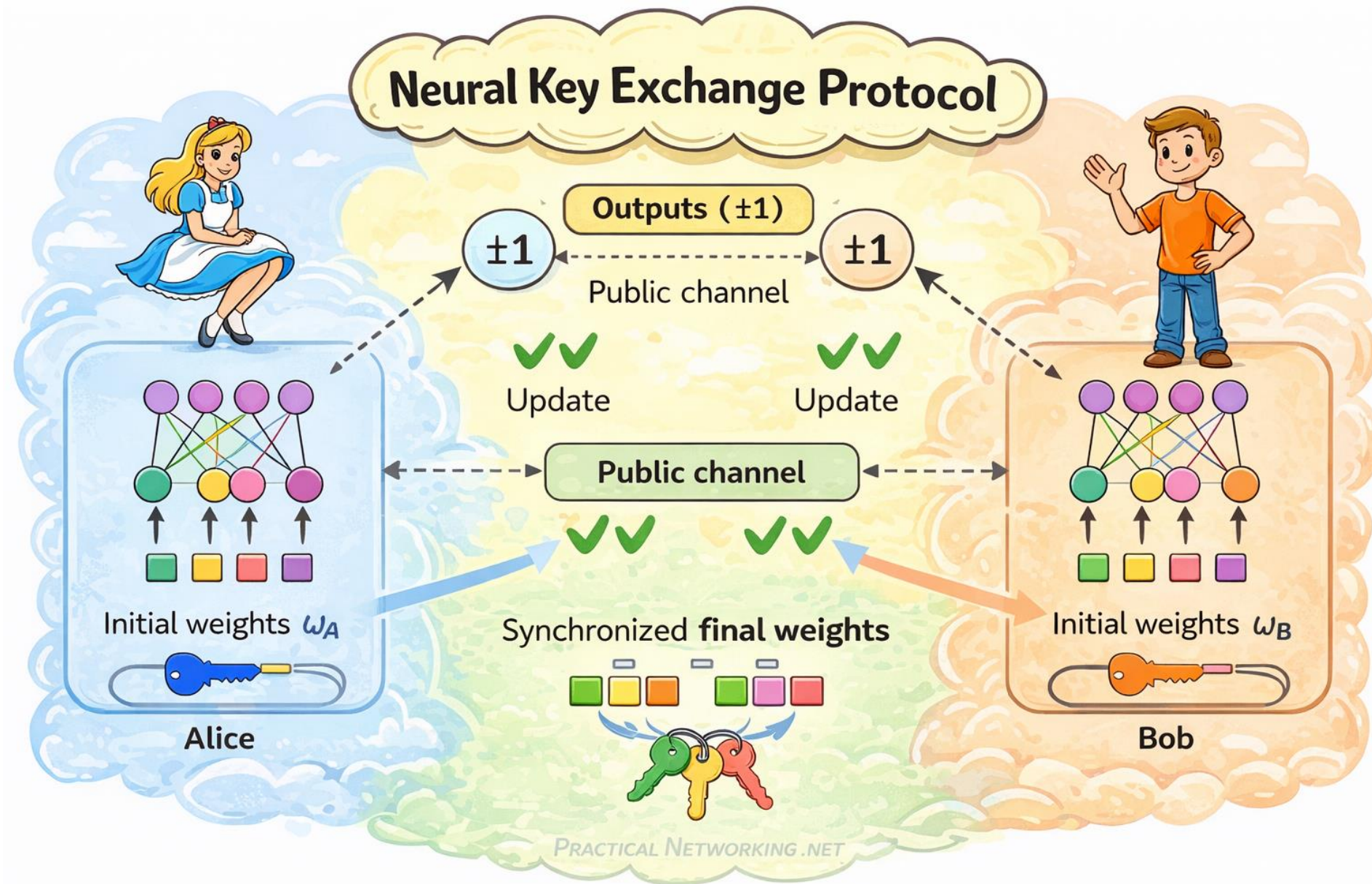
- A cryptographic key exchange method using neural networks.
- Uses synchronization of neural networks instead of traditional math problems.

How It Works:

- Both parties initialize neural networks with random weights.
- They process shared inputs to generate outputs.
- Matching outputs lead to weight updates.
- Iterations result in synchronized networks with identical weights (shared key).



NEURAL
KEY
EXCHANGE



Advantages & Challenges of NKEP

Advantages:

- Potential resilience to quantum attacks.
- Efficient and suitable for resource-constrained devices.
- No reliance on complex mathematical problems.

Challenges:

- Some versions vulnerable to certain attacks.
- Still in research phase with limited real-world use.

MAC - HASH

MESSAGE AUTHENTICATION

- Message (or document) is **authentic** if
 - It is genuine (not altered)
 - came from its alleged source. (not delayed or replayed)
- Message authentication is a **procedure** which verifies that received messages are authentic

Aspects of Message Authentication

We would like to ensure that

- The content of the message has not been changed (**integrity**);
- The source of the message is authentic (**authenticity**);
- The source of the message can be proven by others (**non-repudiation**).

MESSAGE AUTHENTICATION TECHNIQUES

Using conventional message encryption:

- If we assume that only sender and receiver share a secret key then the fact that receiver can successfully decrypt the message means the message has been encrypted by the sender

Without message encryption:

- The message is not encrypted, but special authentication tag is generated and appended to the message. Generation of a tag is a much more efficient procedure than encryption of the message.

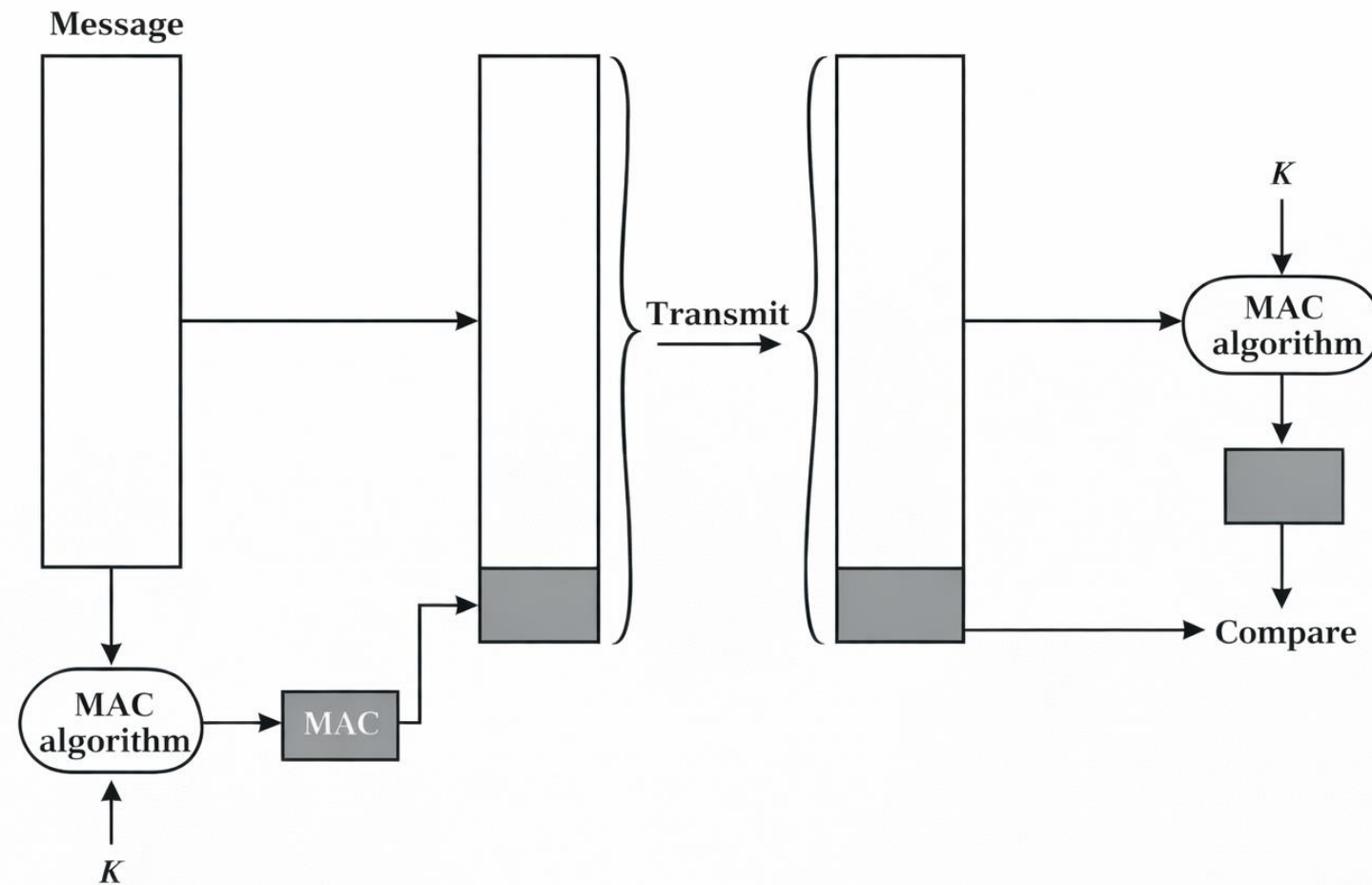
MESSAGE AUTHENTICATION CODE

- Let A and B share a common secret key K
- If A would like to send a message M to B, she calculates a message authentication code MAC of M using the key K

$$\text{MAC} = F(K, M)$$

- Then A appends MAC to M and sends all this to B
- B applies the MAC algorithm to the received message and compares the result with the received MAC

MESSAGE AUTHENTICATION USING MAC



MAC ALGORITHMS

- The process of MAC generation is similar to the encryption;
- The difference is
 - a MAC algorithm need not be reversible
 - easier to implement and less vulnerable to being broken;
- Actually, standard encryption algorithms can be used for MAC generation:
For example, a message may be encrypted with DES and then last 16 or 32 bits of the encrypted text may be used as MAC (too wasteful)
- HMAC-SHA256 is an example of MAC algorithms

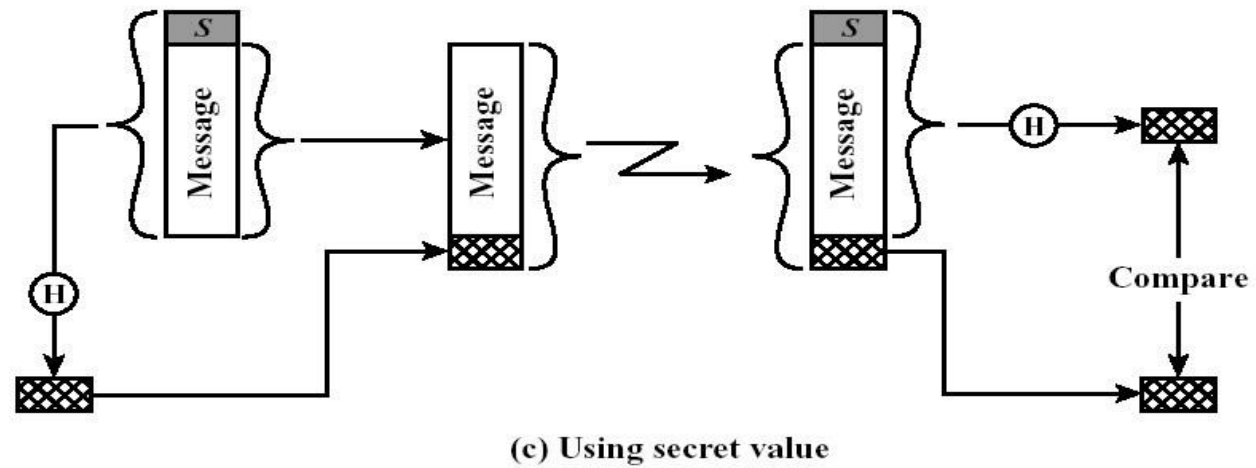
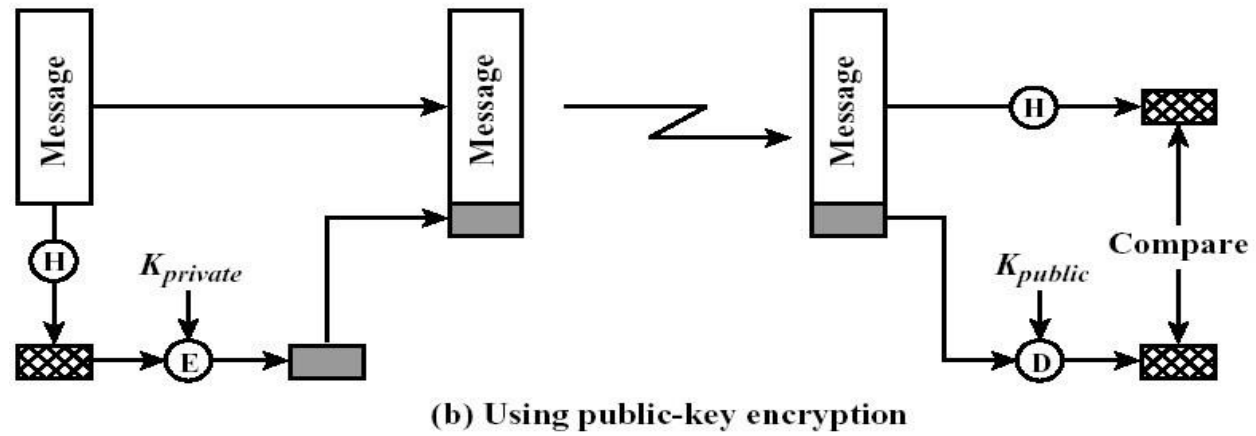
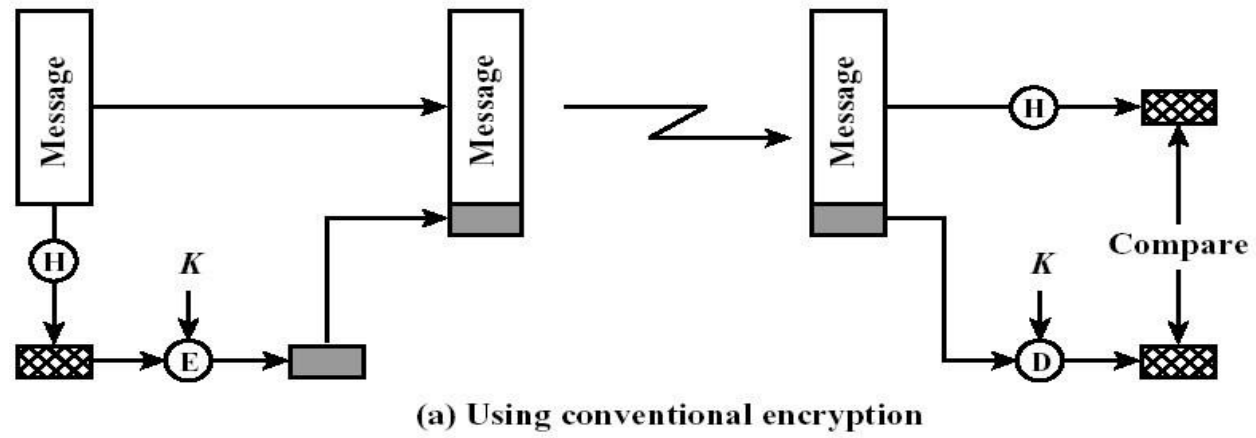
ONE-WAY HASH FUNCTIONS

- Hash functions are important ingredient that can be used for authentication as a part of other mechanisms and protocols
- The main difference with MAC is hash functions don't use a secret key:

$$h = H(M)$$

“One-way” in the name refers to the property of such functions: they are easy to compute, but their reverse functions are very difficult to compute.

METHODS OF AUTHENTICATION USING HASHES



HASH FUNCTION REQUIREMENTS

- To be suitable for message authentication, the hash functions must have ideally the following properties:
- H can be applied to a block of data of any size;
- H produces a fixed-length output;
- $H(x)$ is easy to compute for any given x ;
- For any value h it is very difficult (infeasible) to compute x such that $H(x)=h$ (**one-way property**);
- For any given x , it is very difficult (infeasible) to find y (not equal to x) such that $H(x) = H(y)$; (**weak collision resistance**);
- It is very difficult (infeasible) to find any pair (x,y) such that $H(x) = H(y)$; (**strong collision resistance**).

SIMPLE HASH FUNCTION

- Let the input be a sequence of n-bit blocks
- Then simple hash function does bit-by-bit exclusive-OR (XOR) of every block

	bit 1	bit 2	• • •	bit n
block 1	b_{11}	b_{21}		b_{n1}
block 2	b_{12}	b_{22}		b_{n2}
	•	•	•	•
	•	•	•	•
	•	•	•	•
block m	b_{1m}	b_{2m}		b_{nm}
hash code	C_1	C_2		C_n

S I M P L E H A S H F U N C T I O N

- Simple hash function does not satisfy the weak (and strong) collision property;
- for any message M it is very easy to generate a message M_1 such that $h(M) = h(M_1)$:

Take arbitrary message M_2 , compute $h(M_2) = h_2$, then

Add additional block to M_2 , such that for the resulting M_3 we have $h(M_3) = h(M_1)$.

QUESTIONS

- Which of the aspects:

integrity, authenticity, non-repudiation

the hash functions and discussed schemes do support?

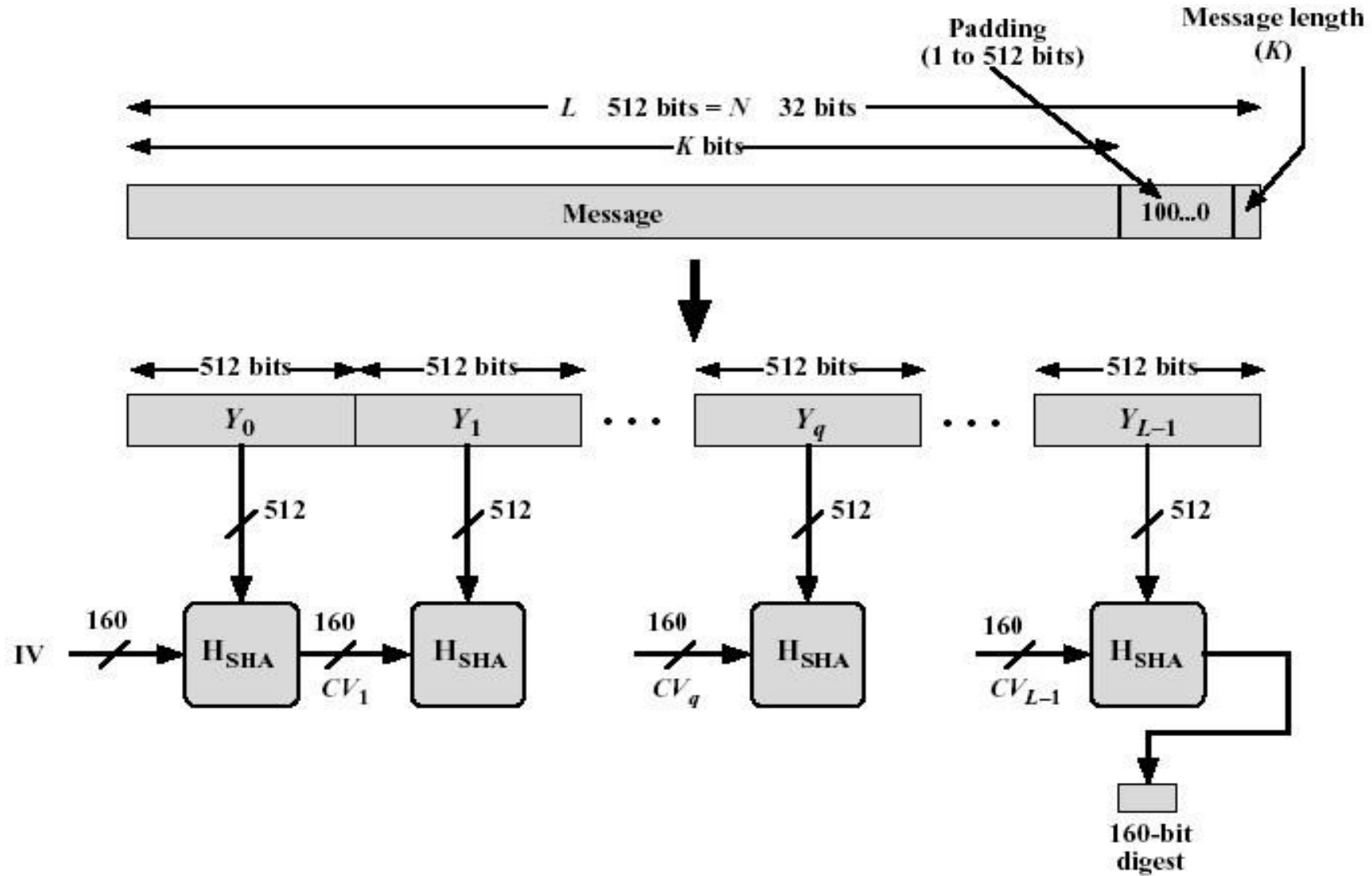
THE SHA - 1 SECURE HASH ALGORITHM

SHA-1 algorithm (1993-1995):

Part of the sample program illustrating password-based encryption (practical sessions)

- Takes as input a message with a maximum length
 - less than 2^{64} bits and produces as output a 160-bit message digest
- The input is processed in 512-bit blocks
- Each bit of the output is computed using all bits of the input (diffusion)

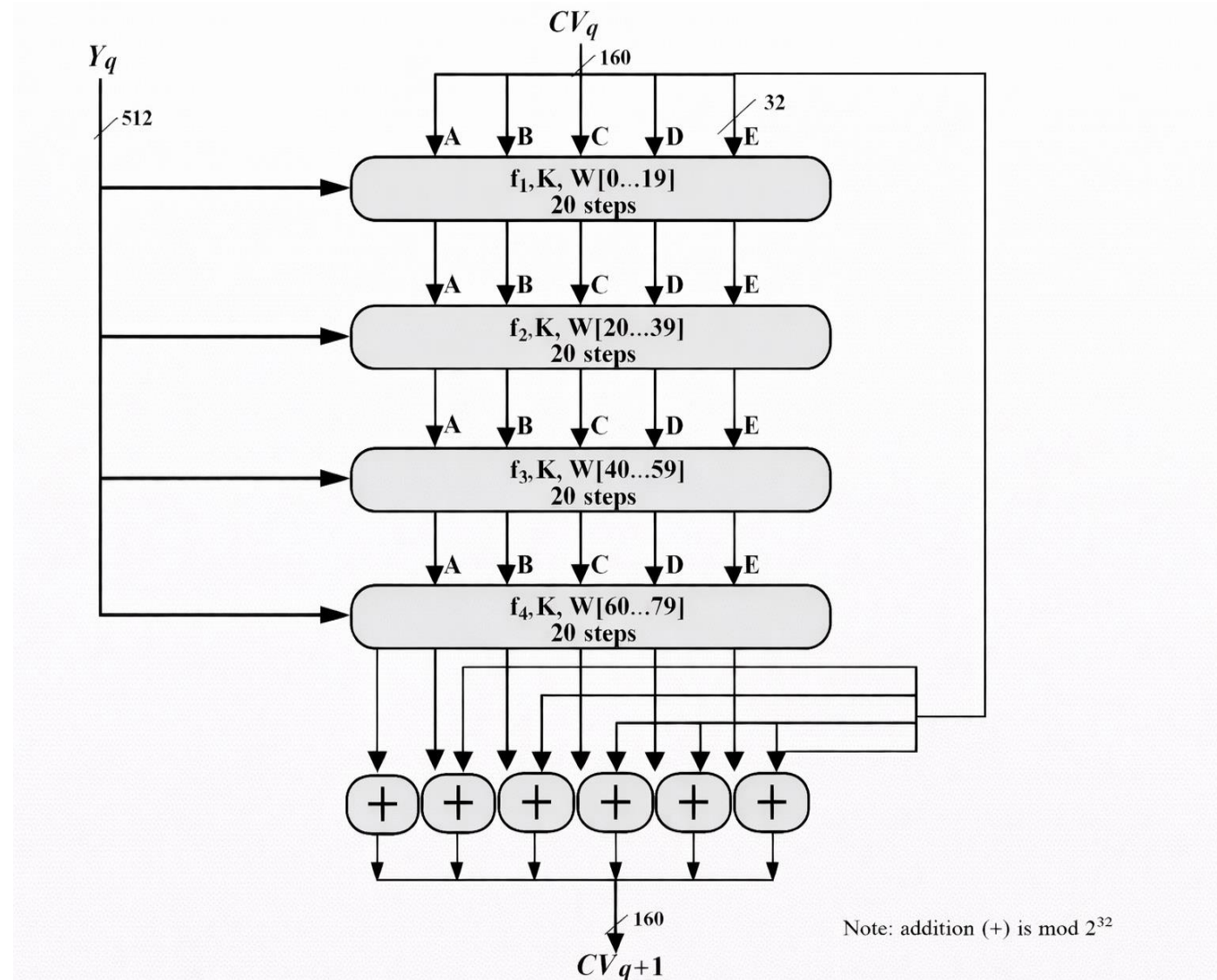
SHA-1 GENERAL SCHEME



SHA-1 PROCESSING A SINGLE BLOCK

The compression function:

- Includes 4 rounds with 20 steps each;
- Each round takes the current 512-bits block and 160-bits buffer value and updates the content of the buffer.



B I R T H D A Y A T T A C K

- Given a function f , how to find x and y such that $f(x) = f(y)$ (collision)
- Repeat until collision found: pick up two (pseudo) random inputs x and y , and check if $f(x) = f(y)$.
- Average number of steps needed $\sim 1.25 * \sqrt{H}$ where H is the number of different values $f(x)$ can take
- For example, if $H = 2^{160}$ then $\sqrt{H} \sim 2^{80}$. So naively birthday attack on SHA-1 would take $\sim 2^{80}$ steps.

PROBLEMS AND SOLUTIONS

- In 2005 a possible mathematical weakness of SHA-1 has been established:
~ 2000 time more efficient than brute force attack was found by Xiaoyun Wang ($\sim 2^{69}$ vs 2^{80})
- Further developments: SHA-2: (SHA-224,-256,-384,-512)
- New competition for the new standard of hash functions by NIST:
 - Deadline for submissions was 31.10.2008
 - New standard SHA-3 is announced a winner on 2nd October 2012; not a replacement, but alternative for SHA-2
- SHA-1 was deprecated as NIST standard in 2011 and disallowed for digital signatures in 2013.

R E C E N T N E W S

- [SHAppening](#), October 2015 “freestart” collision attack
(by M. Stevens, P. Karpman, T. Peyrin)
~US \$2000 of GPU time on EC2, est. ($\sim 2^{57.5}$ steps)
- [SHAttered](#), February 2017, full collision attack
(by Google) $\sim 6,500$ years of CPU time ($\sim 2^{69}$ steps)
- [Chosen prefix attack](#), reported 2020,
SHA-1 is a Shambles, by G. Leurent, T. Peyrin)
<https://sha-mbles.github.io>
Two months using 900 GPU (GTX 1060),
cost 75k USD, estimated rent cost 45k USD ($2^{63.4}$ steps)